

Core Concepts — Neural Networks

Part 02 — 🧠 Deep Learning · 02-deep-learning/notes/01-neural-networks.md

1. From one neuron to a network

A single neuron is exactly what you built in Module 01:

$$\begin{aligned} z &= w^T x + b && \text{(a linear map)} \\ a &= \phi(z) && \text{(a nonlinearity)} \end{aligned}$$

Stack neurons into a **layer**, stack layers into a **network**:

$$\begin{aligned} h_1 &= \phi(W_1 x + b_1) \\ h_2 &= \phi(W_2 h_1 + b_2) \\ \hat{y} &= \text{softmax}(W_3 h_2 + b_3) \end{aligned}$$

Why the nonlinearity is non-negotiable: without ϕ , composing linear maps gives another linear map. A 50-layer linear network has exactly the expressive power of logistic regression. The nonlinearity is the entire reason depth buys you anything.

The universal approximation theorem — and what it doesn't say

A network with one hidden layer and enough units can approximate any continuous function on a compact set to arbitrary accuracy. **What it doesn't say:** how many units ("enough" may be exponential), whether SGD will *find* those weights, or whether the result will generalise. Depth is an efficiency argument, not an expressiveness one: deep networks represent compositional functions with exponentially fewer units than shallow ones.

2. Activation functions

Function	Formula	Range	Use it when
ReLU	$\max(0, z)$	$[0, \infty)$	default for hidden layers; cheap, no vanishing gradient for $z > 0$
Leaky ReLU	$\max(\alpha z, z)$, $\alpha \approx 0.01$	$\mathbb{R}$	fixing "dying ReLU" (neurons stuck at 0 forever)
GELU	$z \cdot \Phi(z)$	$\mathbb{R}$	transformers — smooth, and what BERT/GPT use
SiLU / Swish	$z \cdot \sigma(z)$	$\mathbb{R}$	modern vision nets; close cousin of GELU
SwiGLU	gated Swish(xW) $\odot$ (xV)	$\mathbb{R}$	feed-forward blocks in Llama-class LLMs
Sigmoid	$1/(1+e^{-z})$	$(0,1)$	output only , binary classification
Tanh	$(e^z - e^{-z})/(e^z + e^{-z})$	$(-1,1)$	LSTM gates; zero-centred
Softmax	$e^{z_k} / \sum e^{z_j}$	simplex	output only , multi-class

The rule: ReLU/GELU inside, sigmoid/softmax at the output, and never a sigmoid in a deep hidden stack — its gradient maxes out at 0.25, so ten layers multiply to $0.25^{10} \approx 1e-6$. That's the vanishing gradient problem.

3. Backpropagation = the chain rule on a graph

Forward pass: compute outputs, caching intermediate values. Backward pass: apply the chain rule from loss to parameters, reusing the cache.

For layer l with $z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$ and $a^{(l)} = \phi(z^{(l)})$, define $\delta^{(l)} = \partial J / \partial z^{(l)}$:

```

 $\delta^{(L)} = \nabla_a J \odot \phi'(z^{(L)})$            # at the output layer
 $\delta^{(l)} = (W^{(l+1)\top} \delta^{(l+1)}) \odot \phi'(z^{(l)})$    # propagate backwards
 $\partial J / \partial W^{(l)} = \delta^{(l)} (a^{(l-1)})^\top$ 
 $\partial J / \partial b^{(l)} = \delta^{(l)}$ 

```

Four equations. That's the whole algorithm. Implement them once in `./code/backprop_from_scratch.py` and autograd stops being a black box.

Why softmax + cross-entropy is special

Compute $\partial J / \partial z$ for softmax with cross-entropy and you get the beautifully simple $p - y$. All the exponentials and their derivatives cancel. Every framework fuses these two ops (`cross_entropy` takes *logits*, not probabilities) for exactly this reason — it is both faster and numerically stable.

Common bug: applying softmax yourself and then calling a loss that expects logits. You'll get a model that trains but plateaus early.

4. Initialization: the first thing that breaks

Initialise all weights to zero → every neuron in a layer computes the same thing, receives the same gradient, and stays identical forever. **Symmetry must be broken.**

Scheme	Variance	Pair with
Xavier/Glorot	$2 / (\text{fan_in} + \text{fan_out})$	tanh, sigmoid
He/Kaiming	$2 / \text{fan_in}$	ReLU and friends (default in PyTorch)
Orthogonal	orthogonal matrix	RNNs

The goal is to keep activation variance roughly constant across layers so the signal neither dies nor explodes.

5. The architectures, and what each is really for

Choose the architecture that matches the **structure in your data**.

5.1 MLP — no structure

Fully connected. Use for tabular data and as the "head" on top of any other encoder. Every transformer block contains an MLP; it's where most of the parameters live.

5.2 CNN — spatial/local structure

- **Convolution:** slide a small learnable kernel over the input. Parameters are *shared* across positions.
- Two priors baked in: **locality** (nearby pixels relate) and **translation equivariance** (a cat is a cat anywhere).
- **Pooling / stride:** downsample, grow the receptive field.
- Landmarks: LeNet (1998) → AlexNet (2012, the big bang) → VGG → **ResNet** (2015, residual connections) → EfficientNet → ConvNeXt (2022, CNNs modernised to match transformers).
- Still excellent: small datasets, edge deployment, medical imaging, anything latency-bound.

5.3 RNN / LSTM / GRU — sequential structure

- Maintain a hidden state across time: $h_t = \phi(W_x x_t + W_h h_{t-1} + b)$.
- **LSTM** adds gates (forget/input/output) and a cell state that lets gradients flow far.
- **Fatal flaw:** inherently sequential → no parallelism across time → slow to train on long sequences.
- That flaw is exactly what the transformer removed. **This is the bridge to Module 03.**

5.4 Encoder–decoder + attention

Encode a variable-length input into a representation, decode a variable-length output. Adding *attention* (Bahdanau, 2014) let the decoder look back at all encoder states instead of squeezing everything into one vector. Three years later, "Attention Is All You Need" dropped the recurrence and kept only the attention.

5.5 Autoencoders & representation learning

Compress to a bottleneck and reconstruct. The bottleneck learns a compact representation. **Variational autoencoders (VAEs)** make that latent space probabilistic and sampleable — the direct ancestor of the latent space in Stable Diffusion (Module 05).

6. Embeddings — the idea that unlocks everything later

An embedding is a learned dense vector for a discrete item (word, product, user, node).

vocabulary of 50,000 words → a $50,000 \times 768$ lookup table → each word is a 768-dim vector

Trained end-to-end, similar items end up close together. This single idea powers: - word2vec/GloVe and every LLM's input layer (Module 03/04), - vector databases and semantic search (Module 08), - recommendation systems, and - the "latent space" of generative models (Module 05).

If you only take one concept from Module 02 into the rest of the track, make it this one.

7. Regularization for deep nets

Technique	What it does	Notes
Dropout	randomly zero activations ($p \approx 0.1-0.5$) at train time	scale at train (inverted dropout); disable at eval — <code>model.eval()</code>
Weight decay	L2 shrinkage; use AdamW 's decoupled form	0.01–0.1 typical for transformers
Early stopping	halt on validation plateau	keep the best checkpoint, not the last
Data augmentation	flips, crops, colour jitter, mixup, cutout	the cheapest accuracy you will ever buy
Label smoothing	target 0.9 instead of 1.0	improves calibration, reduces overconfidence
Batch/Layer norm	normalise activations	regularises as a side effect; stabilises training as the main effect

BatchNorm vs. LayerNorm — know the difference

- **BatchNorm** normalises each feature across the *batch*. Great for CNNs; breaks with tiny batches and is awkward for variable-length sequences.
- **LayerNorm** normalises across the *features* of each example independently. Batch-size independent → **this is what transformers use**. Modern LLMs use **RMSNorm**, a cheaper LayerNorm without the mean-centring.

8. Residual connections — why 100+ layers became possible

$$\text{output} = F(x) + x$$

The identity path gives gradients a highway back to early layers, so the effective depth of the gradient signal no longer decays multiplicatively. ResNet (2015) went from ~20 usable layers to 152. **Every transformer block is two residual blocks**. You cannot understand a transformer diagram without this.

9. Transfer learning — the workflow you'll reuse forever

1. Take a model pretrained on a large dataset (ImageNet, or a web-scale text corpus).

2. Replace the task-specific head.
3. **Freeze** the backbone and train the head (fast, works with little data), **or** fine-tune everything with a small learning rate (better, needs more data and care).
4. Optionally unfreeze gradually, with **discriminative learning rates** (lower for earlier layers).

This is the same mental model as LLM fine-tuning in Module 04 — LoRA and adapters are just parameter-efficient versions of step 3.

10. Forward links

Idea here	Where it returns
Embeddings	Modules 03, 04, 08 (vector search)
LayerNorm + residuals	Module 03 — the transformer block
Attention (§5.4)	Module 03 — self-attention
Autoencoder latent space	Module 05 — latent diffusion
Transfer learning	Module 04 — fine-tuning, LoRA
Mixed precision & checkpointing	Module 16 — training infrastructure